

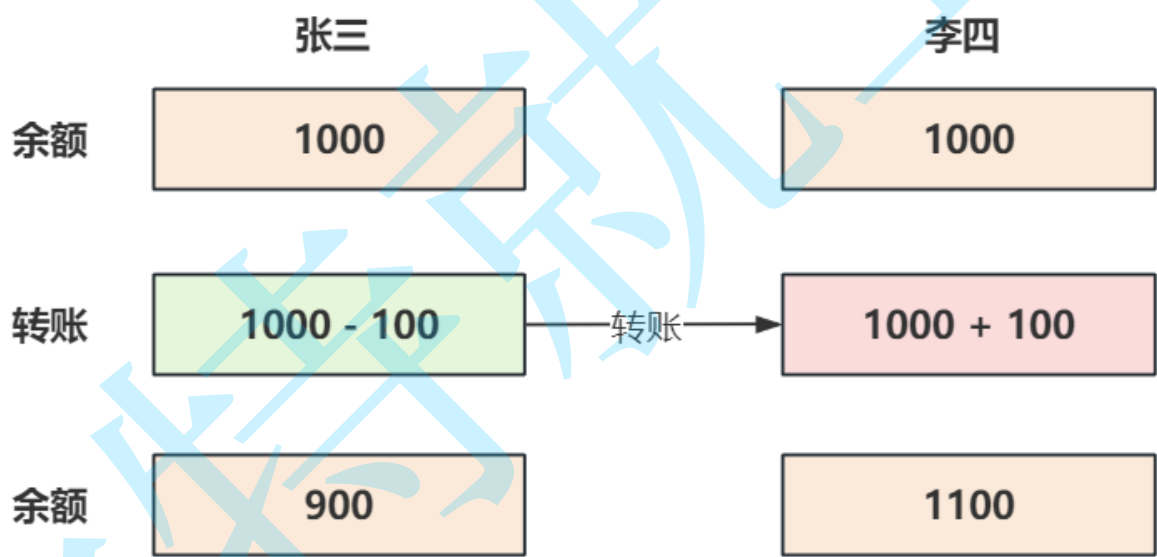
10. 事务

1. 本节目标

- 了解事务的概念与使用场景
- 掌握事务的ACID特性
- 掌握如何使用事务
- 掌握事务的隔离性与隔离级别

2. 什么是事务？

事务把一组SQL语句打包成为一个整体，在这组SQL的执行过程中，要么全部成功，要么全部失败。这组SQL语句可以是一条也可以是多条。来看一个转账的例子，如图：



- 在这个例子中，涉及了两条更新语句：

```
1 # =====账户表=====
2 CREATE TABLE `bank_account` (
3   `id` bigint PRIMARY KEY AUTO_INCREMENT,
4   `name` varchar(255) NOT NULL,      # 姓名
5   `balance` decimal(10, 2) NOT NULL # 余额
6 );
7 INSERT INTO bank_account(`name`, balance) VALUES('张三', 1000);
8 INSERT INTO bank_account(`name`, balance) VALUES('李四', 1000);
9
```

```
10 # =====更新操作=====
11 # 张三余额减少100
12 UPDATE bank_account set balance = balance - 100 where name = '张三';
13 # 李四余额增加100
14 UPDATE bank_account set balance = balance + 100 where name = '李四';
```

- 如果转账成功，应该有以下结果：

1. 张三的账户余额减少 **100**，变成 **900**，李四的账户余额增加了 **100**，变成 **1100**，不能出现张三的余额减少而李四的余额没有增加的情况；
2. 张三和李四在发生转账前后的总额不变，也就是说转账前张三和李四的余额总数为 **1000+1000=2000**，转账后他们的余额总数为 **900+1100=2000**；
3. 转账后的余额结果应当保存到存储介质中，以便以后读取；
4. 还有一点需要注意，在转账的处理过程中张三和李四的余额不能因其他的转账事件而受到干扰；

以上这四点在事务的整个执行过程中必须要得到保证，这也就是事务的 **ACID** 特性

3. 事务的ACID特性

事务的ACID特性指的是 **Atomicity** (原子性)，**Consistency** (一致性)，**Isolation** (隔离性)和 **Durability** (持久性)。

- **Atomicity** (原子性)：一个事务中的所有操作，要么全部成功，要么全部失败，不会出现只执行了一半的情况，如果事务在执行过程中发生错误，会回滚（**Rollback**）到事务开始前的状态，就像这个事务从来没有执行过一样；
- **Consistency** (一致性)：在事务开始之前和事务结束以后，数据库的完整性不会被破坏。这表示写入的数据必须完全符合所有的预设规则，包括数据的精度、关联性以及关于事务执行过程中服务器崩溃后如何恢复；
- **Isolation** (隔离性)：数据库允许多个并发事务同时对数据进行读写和修改，隔离性可以防止多个事务并发执行时由于交叉执行而导致数据的不一致。事务可以指定不同的隔离级别，以权衡在不同的应用场景下数据库性能和安全；
- **Durability** (持久性)：事务处理结束后，对数据的修改将永久的写入存储介质，即便系统故障也不会丢失。

4. 为什么要使用事务？

事务具备的ACID特性，是我们使用事务的原因，在我们日常的业务场景中有大量的需求要用事务来保证。支持事务的数据库能够简化我们的编程模型，不需要我们去考虑各种各样的潜在错误和并发问题，在使用事务过程中，要么提交，要么回滚，不用去考虑网络异常，服务器宕机等其他因素，因此我们经常接触的事务本质上是数据库对 **ACID** 模型的一个实现，是为应用层服务的。

5. 如何使用事务

5.1 查看支持事务的存储引擎

- 要使用事务那么数据库就要支持事务，在MySQL中支持事务的存储引擎是InnoDB，可以通过 `show engines;` 语句查看：

```
mysql> show engines;
```

Engine	Support	Comment	Transactions	XA	Savepoints
MEMORY	YES	Hash based, stored in memory, useful for temporary tables	NO	NO	NO
MRG_MYISAM	YES	Collection of identical MyISAM tables	NO	NO	NO
CSV	YES	CSV storage engine	NO	NO	NO
FEDERATED	NO	Federated MySQL storage engine	NULL	NULL	NULL
PERFORMANCE_SCHEMA	YES	Performance Schema	NO	NO	NO
MyISAM	YES	MyISAM storage engine	NO	NO	NO
InnoDB	DEFAULT	Supports transactions, row-level locking, and foreign keys	YES	YES	YES
ndbinfo	NO	MySQL Cluster system information storage engine	NULL	NULL	NULL
BLACKHOLE	YES	/dev/null storage engine (anything you write to it disappears)	NO	NO	NO
ARCHIVE	YES	Archive storage engine	NO	NO	NO
ndbcluster	NO	Clustered, fault-tolerant tables	NULL	NULL	NULL

11 rows in set (0.00 sec)

5.2 语法

- 通过以下语句可以完成对事务的控制：

```
1 # 开始一个新的事务
2 START TRANSACTION;
3 # 或
4 BEGIN;
5
6 # 提交当前事务，并对更改持久化保存
7 COMMIT;
8
9 # 回滚当前事务，取消其更改
10 ROLLBACK;
```

- `START TRANSACTION` 或 `BEGIN` 开始一个新的事务；
- `COMMIT` 提交当前事务，并对更改持久化保存；
- `ROLLBACK` 回滚当前事务，取消其更改；
- 无论提交还是回滚，事务都会关闭

5.3 开启一个事务，执行修改后回滚

```
1 # 开启事务
2 mysql> START TRANSACTION;
3 Query OK, 0 rows affected (0.00 sec)
4
```

```

5 # 在修改之前查看表中的数据
6 mysql> select * from bank_account;
7 +-----+-----+-----+
8 | id | name | balance |
9 +-----+-----+-----+
10 | 1 | 张三 | 1000.00 |
11 | 2 | 李四 | 1000.00 |
12 +-----+-----+-----+
13 2 rows in set (0.00 sec)
14
15 # 张三余额减少100
16 mysql> UPDATE bank_account set balance = balance - 100 where name = '张三';
17 Query OK, 1 row affected (0.00 sec)
18 Rows matched: 1 Changed: 1 Warnings: 0
19
20 # 李四余额增加100
21 mysql> UPDATE bank_account set balance = balance + 100 where name = '李四';
22 Query OK, 1 row affected (0.00 sec)
23 Rows matched: 1 Changed: 1 Warnings: 0
24
25 # 在修改之后，提交之前查看表中的数据，余额已经被修改
26 mysql> select * from bank_account;
27 +-----+-----+-----+
28 | id | name | balance |
29 +-----+-----+-----+
30 | 1 | 张三 | 900.00 |
31 | 2 | 李四 | 1100.00 |
32 +-----+-----+-----+
33 2 rows in set (0.00 sec)
34
35 # 回滚事务
36 mysql> ROLLBACK;
37 Query OK, 0 rows affected (0.00 sec)
38
39 # 再查询发现修改没有生效
40 mysql> select * from bank_account;
41 +-----+-----+-----+
42 | id | name | balance |
43 +-----+-----+-----+
44 | 1 | 张三 | 1000.00 |
45 | 2 | 李四 | 1000.00 |
46 +-----+-----+-----+
47 2 rows in set (0.00 sec)

```

5.4 开启一个事务，执行修改后提交

```
1 # 开启事务
2 mysql> BEGIN;
3 Query OK, 0 rows affected (0.00 sec)
4
5 # 在修改之前查看表中的数据
6 mysql> select * from bank_account;
7 +-----+-----+-----+
8 | id | name | balance |
9 +-----+-----+-----+
10 | 1 | 张三 | 1000.00 |
11 | 2 | 李四 | 1000.00 |
12 +-----+-----+-----+
13 2 rows in set (0.00 sec)
14
15 # 张三余额减少100
16 mysql> UPDATE bank_account set balance = balance - 100 where name = '张三';
17 Query OK, 1 row affected (0.00 sec)
18 Rows matched: 1 Changed: 1 Warnings: 0
19
20 # 李四余额增加100
21 mysql> UPDATE bank_account set balance = balance + 100 where name = '李四';
22 Query OK, 1 row affected (0.00 sec)
23 Rows matched: 1 Changed: 1 Warnings: 0
24
25 # 在修改之后，提交之前查看表中的数据，余额已经被修改
26 mysql> select * from bank_account;
27 +-----+-----+-----+
28 | id | name | balance |
29 +-----+-----+-----+
30 | 1 | 张三 | 900.00 |
31 | 2 | 李四 | 1100.00 |
32 +-----+-----+-----+
33 2 rows in set (0.00 sec)
34
35 # 提交事务
36 mysql> COMMIT;
37 Query OK, 0 rows affected (0.01 sec)
38
39 # 再查询发现数据已被修改，说明数据已经持久化到磁盘
40 mysql> select * from bank_account;
41 +-----+-----+-----+
42 | id | name | balance |
43 +-----+-----+-----+
44 | 1 | 张三 | 900.00 |
45 | 2 | 李四 | 1100.00 |
46 +-----+-----+-----+
47 2 rows in set (0.00 sec)
```

5.5 保存点

在事务执行的过程中设置保存点，回滚时指定保存点可以把数据恢复到保存点的状态

```
1 # 开启事务
2 mysql> START TRANSACTION;
3 Query OK, 0 rows affected (0.00 sec)
4
5 # 在修改之前查看表中的数据
6 mysql> select * from bank_account;
7 +-----+-----+
8 | id | name | balance |
9 +-----+-----+
10 | 1 | 张三 | 900.00 |
11 | 2 | 李四 | 1100.00 |
12 +-----+-----+
13 2 rows in set (0.00 sec)
14
15 # 张三余额减少100
16 mysql> UPDATE bank_account set balance = balance - 100 where name = '张三';
17 Query OK, 1 row affected (0.00 sec)
18 Rows matched: 1 Changed: 1 Warnings: 0
19
20 # 李四余额增加100
21 mysql> UPDATE bank_account set balance = balance + 100 where name = '李四';
22 Query OK, 1 row affected (0.00 sec)
23 Rows matched: 1 Changed: 1 Warnings: 0
24
25 # 余额已经被修改
26 mysql> select * from bank_account;
27 +-----+-----+
28 | id | name | balance |
29 +-----+-----+
30 | 1 | 张三 | 800.00 |
31 | 2 | 李四 | 1200.00 |
32 +-----+-----+
33 2 rows in set (0.00 sec)
34
35 # 设置保存点
36 mysql> SAVEPOINT savepoint1;
37 Query OK, 0 rows affected (0.01 sec)
38
39 # 再次执行，张三余额减少100
40 mysql> UPDATE bank_account set balance = balance - 100 where name = '张三';
41 Query OK, 1 row affected (0.00 sec)
```

```

42 Rows matched: 1  Changed: 1  Warnings: 0
43
44 # 再次执行, 李四余额增加100
45 mysql> UPDATE bank_accountset balance = balance + 100 where name = '李四';
46 Query OK, 1 row affected (0.00 sec)
47 Rows matched: 1  Changed: 1  Warnings: 0
48
49 # 余额已经被修改
50 mysql> select * from bank_account;
51 +-----+-----+-----+
52 | id | name  | balance |
53 +-----+-----+-----+
54 | 1 | 张三  | 700.00 |
55 | 2 | 李四  | 1300.00 |
56 +-----+-----+-----+
57 2 rows in set (0.00 sec)
58
59 # 设置第二个保存点
60 mysql> SAVEPOINT savepoint2;
61 Query OK, 0 rows affected (0.00 sec)
62
63 # 插入一条新记录
64 mysql> insert into bank_account values (null, '王五', 1000);
65 Query OK, 1 row affected (0.01 sec)
66
67 # 查询结果, 新记录写入成功
68 mysql> select * from bank_account;
69 +-----+-----+-----+
70 | id | name  | balance |
71 +-----+-----+-----+
72 | 1 | 张三  | 700.00 |
73 | 2 | 李四  | 1300.00 |
74 | 3 | 王五  | 1000.00 |
75 +-----+-----+-----+
76 3 rows in set (0.00 sec)
77
78 # 回滚到第二个保存点
79 mysql> ROLLBACK TO savepoint2;
80 Query OK, 0 rows affected (0.00 sec)
81
82 # 回滚成功
83 mysql> select * from bank_account;
84 +-----+-----+-----+
85 | id | name  | balance |
86 +-----+-----+-----+
87 | 1 | 张三  | 700.00 |
88 | 2 | 李四  | 1300.00 |

```

```

89 +-----+
90 2 rows in set (0.00 sec)
91
92 # 回滚到第一个保存点
93 mysql> ROLLBACK TO savepoint1;
94 Query OK, 0 rows affected (0.00 sec)
95
96 # 回滚成功
97 mysql> select * from bank_account;
98 +-----+
99 | id | name  | balance |
100 +-----+
101 | 1  | 张三  | 800.00  |
102 | 2  | 李四  | 1200.00 |
103 +-----+
104 2 rows in set (0.00 sec)
105
106 # 回滚时不指定保存点，直接回滚到事务开始时的原始状态，事务关闭
107 mysql> ROLLBACK;
108 Query OK, 0 rows affected (0.01 sec)
109
110 # 原始状态
111 mysql> select * from bank_account;
112 +-----+
113 | id | name  | balance |
114 +-----+
115 | 1  | 张三  | 900.00  |
116 | 2  | 李四  | 1100.00 |
117 +-----+
118 2 rows in set (0.00 sec)

```

5.6 自动/手动提交事务

- 默认情况下，MySQL是自动提交事务的，也就是说我们执行的每个修改操作，比如插入、更新和删除，都会自动开启一个事务并在语句执行完成之后自动提交，发生异常时自动回滚。
- 查看当前事务是否自动提交可以使用以下语句

```

1 mysql> show variables like 'autocommit';
2 +-----+
3 | Variable_name | Value |
4 +-----+
5 | autocommit    | ON    | # ON 表示自动提交开启
6 +-----+
7 1 row in set, 1 warning (0.04 sec)

```


- 可以通过以下语句设置事务为自动或手动提交

```
1 # 设置事务自动提交
2 mysql> SET AUTOCOMMIT=1;      # 方式一
3 mysql> SET AUTOCOMMIT=ON;     # 方式二
4
5 # 设置事务手动提交
6 mysql> SET AUTOCOMMIT=0;      # 方式一
7 mysql> SET AUTOCOMMIT=OFF;    # 方式二
```

注意：

- 只要使用 `START TRANSACTION` 或 `BEGIN` 开启事务，必须要通过 `COMMIT` 提交才会持久化，与是否设置 `SET autocommit` 无关。
- 手动提交模式下，不用显示开启事务，执行修改操作后，提交或回滚事务时直接使用 `commit` 或 `rollback`
- 已提交的事务不能回滚

6. 事务的隔离性和隔离级别

6.1 什么是隔离性

MySQL服务可以同时被多个客户端访问，每个客户端执行的DML语句以事务为基本单位，那么不同的客户端在对同一张表中的同一条数据进行修改的时候就可能出现相互影响的情况，为了保证不同的事务之间在执行的过程中不受影响，那么事务之间就需要相互隔离，这种特性就是**隔离性**。

6.2 隔离级别

事务具有隔离性，那么如何实现事务之间的隔离？隔离到什么程度？如何保证数据安全的同时也要兼顾性能？这都是要思考的问题。

事务间不同程度的隔离，称为**事务的隔离级别**；不同的隔离级别在性能和安全方面做了取舍，有的隔离级别注重并发性，有的注重安全性，有的则是并发和安全适中；在MySQL的InnoDB引擎中事务的隔离级别有**四种**，分别是：

- `READ UNCOMMITTED`，读未提交
- `READ COMMITTED`，读已提交
- `REPEATABLE READ`，可重复读(默认)
- `SERIALIZABLE`，串行化

6.3 查看和设置隔离级别

- 事务的隔离级别分为全局作用域和会话作用域，查看不同作用域事务的隔离级别，可以使用以下方式：

```
1 # 全局作用域
2 SELECT @@GLOBAL.transaction_isolation;
3 # 会话作用域
4 SELECT @@SESSION.transaction_isolation;
5 # 可以看到默认的事务隔离级别是REPEATABLE-READ(可重复读)
6 +-----+
7 | @@SESSION.transaction_isolation |
8 +-----+
9 | REPEATABLE-READ                  | # 默认是可重复读
10 +-----+
11 1 row in set (0.00 sec)
```

- 设置事务的隔离级别和访问模式，可以使用以下语法：

```
1 # 通过GLOBAL|SESSION分别指定不同作用域的事务隔离级别
2 SET [GLOBAL|SESSION] TRANSACTION ISOLATION LEVEL level|access_mode;
3
4 # 隔离级别
5 level: {
6     REPEATABLE READ # 可重复读
7     | READ COMMITTED # 读已提交
8     | READ UNCOMMITTED # 读未提交
9     | SERIALIZABLE # 串行化
10 }
11
12 # 访问模式
13 access_mode: {
14     READ WRITE # 表示事务可以对数据进行读写
15     | READ ONLY # 表示事务是只读，不能对数据进行读写
16 }
17
18 # 示例
19 # 设置全局事务隔离级别为串行化，后续所有事务生效，不影响当前事务
20 SET GLOBAL TRANSACTION ISOLATION LEVEL SERIALIZABLE;
21 # 设置会话事务隔离级别为串行化，当前会话后续的所有事务生效，不影响当前事务，可以在任何时候执行
22 SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE;
23 # 如果不指定任何作用域，设置只针对下一个事务，随后的事务恢复之前的隔离级别
```

```

1 # 方式一
2 SET GLOBAL transaction_isolation = 'SERIALIZABLE';
3 # 注意使用SET语法时有空格要用"-"代替
4 SET SESSION transaction_isolation = 'REPEATABLE-READ';
5
6 # 方式二
7 SET @@GLOBAL.transaction_isolation='SERIALIZABLE';
8 # 注意使用SET语法时有空格要用"-"代替
9 SET @@SESSION.transaction_isolation='REPEATABLE-READ';

```

6.4 不同隔离级别存在的问题

6.4.1 READ UNCOMMITTED - 读未提交与脏读

6.4.1.1 存在问题

出现在事务的 **READ UNCOMMITTED** 隔离级别下，由于在读取数据时不做任何限制，所以并发性能很高，但是会出现大量的数据安全问题，比如在事务A中执行了一条 **INSERT** 语句，在没有执行 **COMMIT** 的情况下，会在事务B中被读取到，此时如果事务A执行回滚操作，那么事务B中读取到事务A写入的数据将没有意义，我们把这个现象叫做 **"脏读"**。

6.4.1.2 问题重现

- 在一个客户端A中先设置全局事务隔离级别为 **READ UNCOMMITTED** 读未提交:

```

1 # 设置隔离级别为READ UNCOMMITTED读未提交
2 mysql> SET GLOBAL TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
3 Query OK, 0 rows affected (0.00 sec)
4
5 # 查看设置是否生效
6 mysql> SELECT @@GLOBAL.transaction_isolation;
7 +-----+
8 | @@GLOBAL.transaction_isolation |
9 +-----+
10 | READ-UNCOMMITTED                | # 已生效
11 +-----+
12 1 row in set (0.00 sec)
13

```

- 打开另一个客户端B并确认隔离级别

```

1 # 查看设置是否生效
2 mysql> SELECT @@GLOBAL.transaction_isolation;
3 +-----+
4 | @@GLOBAL.transaction_isolation |
5 +-----+
6 | READ-UNCOMMITTED                | # 已生效
7 +-----+
8 1 row in set (0.00 sec)
9

```

- 在不同的客户端中执行事务

会话A开启事务A

```

1 # 选择数据库
2 use test_db;
3 # 开启事务
4 START TRANSACTION;

```

```

1 # 写入一条新数据
2 insert into bank_account values
3 (null, '王五', 2000);
4

```

```

1 # 查询结果，
2 select * from bank_account;
3 +-----+
4 | id | name | balance |
5 +-----+
6 | 1 | 张三 | 800.00 |
7 | 2 | 李四 | 1100.00 |
8 | 5 | 王五 | 2000.00 |
9 +-----+
10 3 rows in set (0.00 sec)
11
12 # 新记录已写入，但是此时事务A并没有提交

```

会话B开启事务B

```
1 # 选择数据库
2 use test_db;
3 # 开启事务
4 START TRANSACTION;
```

```
1 # 查询结果
2 select * from bank_account;
3 +-----+-----+
4 | id | name | balance |
5 +-----+-----+
6 | 1 | 张三 | 800.00 |
7 | 2 | 李四 | 1100.00 |
8 | 5 | 王五 | 2000.00 |
9 +-----+-----+
10 3 rows in set (0.00 sec)
11
12 # 发现查到了事务A没有提交的数据
```

```
1 # 回滚
2 rollback;
3
4 # 查询结果，数据正常回滚
5 select * from bank_account;
6 +-----+-----+
7 | id | name | balance |
8 +-----+-----+
9 | 1 | 张三 | 800.00 |
10 | 2 | 李四 | 1100.00 |
11 +-----+-----+
12 2 rows in set (0.00 sec)
```

```
1 # 再次查询，刚才“王五”这条记录不存在了
2 select * from bank_account;
3 +-----+-----+
4 | id | name | balance |
5 +-----+-----+
6 | 1 | 张三 | 800.00 |
7 | 2 | 李四 | 1100.00 |
```

```
8 +-----+-----+-----+
9 2 rows in set (0.00 sec)
```

- 由于 **READ UNCOMMITTED 读未提交** 会出现"脏读"现象，在正常的业务中出现这种问题会产生非常危重后果，所以正常情况下应该避免使用 **READ UNCOMMITTED 读未提交** 这种的隔离级别。

6.4.2 READ COMMITTED - 读已提交与不可重复读

6.4.2.1 存在问题

为了解决脏读问题，可以把事务的隔离级别设置为 **READ COMMITTED**，这时事务只能读到了其他事务提交之后的数据，但会出现不可重复读的问题，比如事务A先对某条数据进行了查询，之后事务B对这条数据进行了修改，并且提交(**COMMIT**)事务，事务A再对这条数据进行查询时，得到了事务B修改之后的结果，这导致了事务A在同一个事务中以相同的条件查询得到了不同的值，这个现象要"不可重复读"。

6.4.2.2 问题重现

- 在一个客户端A中先设置全局事务隔离级别为 **READ COMMITTED 读未提交**:

```
1 # 设置隔离级别为READ COMMITTED读未提交
2 mysql> SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
3 Query OK, 0 rows affected (0.00 sec)
4
5 # 查看设置是否生效
6 mysql> SELECT @@GLOBAL.transaction_isolation;
7 +-----+
8 | @@GLOBAL.transaction_isolation |
9 +-----+
10 | READ-COMMITTED                  | # 已生效
11 +-----+
12 1 row in set (0.00 sec)
13
```

- 打开另一个客户端B并确认隔离级别

```
1 # 查看设置是否生效
2 mysql> SELECT @@GLOBAL.transaction_isolation;
3 +-----+
4 | @@GLOBAL.transaction_isolation |
5 +-----+
6 | READ-COMMITTED                  | # 已生效
```

```
7 +-----+
8 1 row in set (0.00 sec)
9
```

- 在不同的客户端中执行事务

会话A开启事务A	会话B开启事务B
	<pre>1 # 选择数据库 2 use test_db; 3 # 写入一条新测试数据 4 insert into bank_account values 5 (null, '王五', 2000); 6 Query OK, 1 row affected (0.00 sec)</pre>
	<pre>1 # 开启事务 2 START TRANSACTION; 3 4 # 查询王五的记录，余额是2000 5 select * from bank_account where name='王五'; 6 +-----+ 7 id name balance 8 +-----+ 9 6 王五 2000.00 10 +-----+ 11 1 row in set (0.01 sec)</pre>
<pre>1 # 选择数据库 2 use test_db; 3 # 开启事务 4 START TRANSACTION;</pre>	
<pre>1 # 修改王五的余额为1000 2 update bank_account set balance=1000</pre>	

```
3 where name = '王五';
4 Query OK, 1 row affected (0.00 sec)
```

```
1 # 提交事务
2 commit;
```

```
1 # 此时事务并没有提交或回滚
2 # 再次查询王五的记录发现余额变成了
  1000
3 # 与上一个查询结果不一致
4 select * from bank_account
  where name='王五';
5 +-----+
6 | id | name | balance |
7 +-----+
8 |  6 | 王五 | 1000.00 | # 出现
   问题
9 +-----+
10 1 row in set (0.00 sec)
```

6.4.3 REPEATABLE READ - 可重复读与幻读

6.4.3.1 存在问题

为了解决不可重复读问题，可以把事务的隔离级别设置为 **REPEATABLE READ**，这时同一个事务中读取的数据在任何时候都是相同的结果，但还会出现一个问题，事务A查询了一个区间的记录得到**结果集A**，事务B向这个区间的间隙中写入了一条记录并提交，事务A再查询这个区间的结果集时会查到事务B新写入的记录得到**结果集B**，两次查询的**结果集不一致**，这个现象就是"幻读"。

MySQL的InnoDB存储引擎使用了**Next-Key**锁解决了大部分幻读问题

6.4.3.2 问题重现

- 由于 **REPEATABLE READ** 隔离级别默认使用了 **Next-Key** 锁，为了重现幻读问题，我们把隔离级回退到更新时只加了排他锁的 **READ COMMITTED**。

```
1 # 设置隔离级别为READ COMMITTED读未提交
2 mysql> SET GLOBAL TRANSACTION ISOLATION LEVEL READ COMMITTED;
3 Query OK, 0 rows affected (0.00 sec)
4
```



```

5 # 查看设置是否生效
6 mysql> SELECT @@GLOBAL.transaction_isolation;
7 +-----+
8 | @@GLOBAL.transaction_isolation |
9 +-----+
10 | READ-COMMITTED                  | # 已生效
11 +-----+
12 1 row in set (0.00 sec)

```

- 在不同的客户端中执行事务

会话A开启事务A	会话B开启事务B
	<pre> 1 # 选择数据库 2 use test_db; 3 # 开启事务 4 START TRANSACTION; </pre>
	<pre> 1 # 更新五五的余额，使该记录加排他锁 2 update account set bank_account=2000 3 where name='王五'; 4 Query OK, 1 row affected (0.01 sec) </pre>
	<pre> 1 # 查询结果集，更新成功 2 select * from bank_account; 3 +-----+ 4 id name balance 5 +-----+ 6 1 张三 800.00 7 2 李四 1100.00 8 6 王五 2000.00 # 更新成功 9 +-----+ 10 3 rows in set (0.00 sec) </pre>

```
1 # 选择数据库
2 use test_db;
3 # 开启事务
4 START TRANSACTION;
```

```
1 # 在李“四与”和“王五”之间的间隙写入
2 # 一条数据“赵六”
3 insert into bank_account
4 values (3, '赵六', 5000);
```

```
1 # 查询结果集，写入成功
2 select * from bank_account;
3 +-----+-----+-----+
4 | id | name | balance |
5 +-----+-----+-----+
6 | 1 | 张三 | 800.00 |
7 | 2 | 李四 | 1100.00 |
8 | 3 | 赵六 | 5000.00 |
9 | 6 | 王五 | 1000.00 |
10 +-----+-----+-----+
11 4 rows in set (0.01 sec)
12
```

```
1 # 提交事务
2 commit;
```

```
1 # 查询结果集，
2 # 发现比上一次查询的结果集
3 # 多出了其他事务写入的数据
4 select * from bank_account;
5 +-----+-----+-----+
6 | id | name | balance |
7 +-----+-----+-----+
8 | 1 | 张三 | 800.00 |
9 | 2 | 李四 | 1100.00 |
```

	10 3 赵六 5000.00 # 幻像行 11 6 王五 2000.00 12 +-----+-----+-----+ 13 4 rows in set (0.01 sec)
	1 # 提交事务 2 commit;

把隔离级别设置为REPEATABLE-READ后，在ID的间隙中插入新数据观察现象，比如插入ID = 4的记录

6.4.4 SERIALIZABLE - 串行化

进一步提升事务的隔离级别到 **SERIALIZABLE**，此时所有事务串行执行，可以解决所有并发中的安全问题。

6.5 不同隔离级别的性能与安全

高 ↑ 并发性能	隔离级别	脏读	不可重复读	幻读	低 ↓ 隔离力度 (安全性)
	READ UNCOMMITTED	存在	存在	存在	
	READ COMMITTED	解决	存在	存在	
	REPEATABLE READ	解决	解决	存在	
低	SERIALIZABLE	解决	解决	解决	高

- InnoDB存储引擎事务隔离性以及相关的隔离级别是由锁和MVCC机制配合实现的，关于锁与MVCC的相关内容和原子性，持久性，一致性的相关实现原理可以参阅MySQL进阶中事务章节